

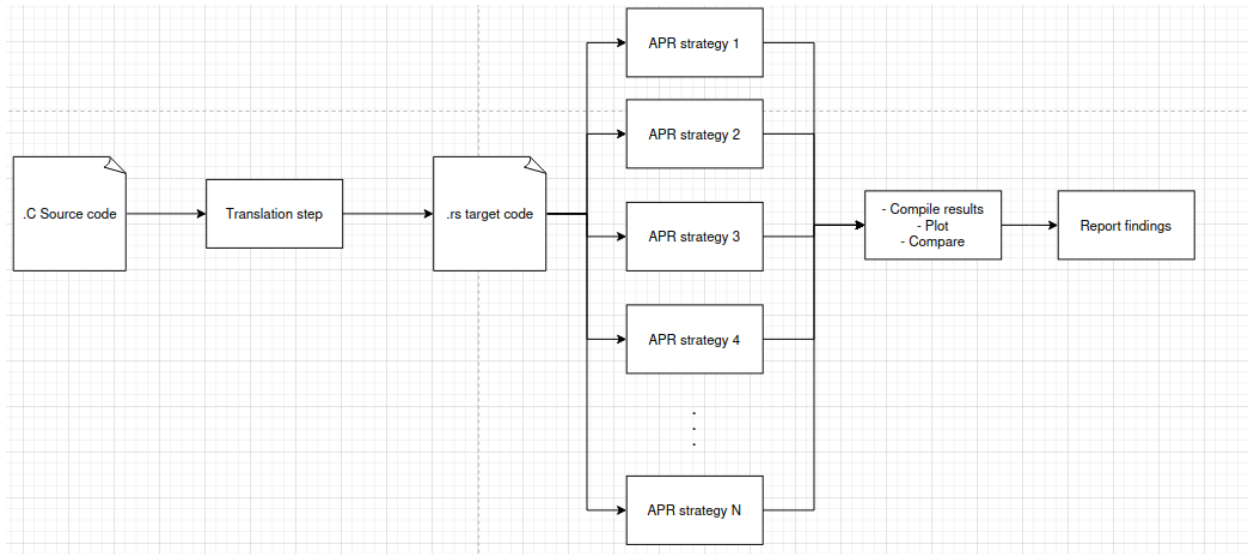
Experimental setup for APR

This document will propose an experimental setup for my APR thesis. It will begin by just briefly explaining all of the assumptions given and then proceed to propose some of the APR strategies that i can deploy to compare with each other for my actual experiment. I will also show some of the datasets that i have prepared for now as well as metrics that i think would be interesting to measure. You can check out the repo for my project here:

<https://github.com/kemalcikota113/master-thesis>

Assumptions and experimental setup

In order to make an experiment that focuses on the APR stage and not the translation step, for each dataset found in .C, the translation will only be ran once to generate a sufficient translation. The criteria to be deemed sufficient will be that the function names exist in the translated code and that there is some kind of logic that tries to replicate the behavior of the .C code. For now, the translation will not be evaluated in any quantitative way, only by eyeballing it. The *same* Rust target code will then undergo the different APR strategies separately. I do this because i want the different APR strategies to be exposed to the same code in order to make the experiment as fair as possible. The APR strategies will track different metrics but one of the leading one that i am currently most interested in is error count. These different error counts could then be plotted over the amount of iterations done to get a progression plot of how well the code is translated by the different strategies. This same process will then be repeated for each dataset that i have in C code. This is a basic concept of the experiment pipeline that i have set up to illustrate the flow of the experiment. This is also assuming that the codebase that i have pulled from the GH repos have been cleaned as well. I provide exact documentation on what i have filtered out and the test logs in my own repo.



Datasets

These are some of the datasets that i have found this far that i think hold some kind of industrial relevancy and also are interesting to see if they can be translated in to Rust code. The datasets that i have picked are mostly written in just plain and raw C and are respected implementations judging from the amount of stars they have on GitHub. I will give the links to them as well as a short description of each one.

- sds

- **Link:** <https://github.com/antirez/sds>

- **What it does:** A high-performance dynamic string library used in Redis-like systems.

- **Interesting C features:** pointer arithmetic, custom memory-layout headers before string pointers, macro-heavy low-level utilities, variadic formatting functions.

- **Size (cleaned set):** sds.c ~1328 LOC, sds.h 274 LOC, sdsalloc.h 42 LOC.

- cJSON

- **Link:** <https://github.com/DaveGamble/cJSON>

- **What it does:** Lightweight JSON parser/printer in ANSI C.

- **Interesting C features:** recursive parsing, linked-list style object/array representation, custom allocator hooks, extensive pointer-based traversal and mutation.

- **Size (cleaned set):** cJSON.c ~3191 LOC, cJSON_Utils.c 1481 LOC, headers 394 LOC combined.

- http-parser

- **Link:** <https://github.com/nodejs/http-parser>

- **What it does:** Event/callback-driven HTTP request/response parser designed for high-performance servers.

- **Interesting C features:** state-machine parsing, callback-driven architecture, zero-allocation parsing style, tight control-flow logic and enum/state transitions.

- **Size (cleaned set):** http_parser.c ~2575 LOC, http_parser.h ~449 LOC.

- **kilo**

- **Link:** <https://github.com/antirez/kilo>

- **What it does:** Minimal terminal text editor (single-file C program).

- **Interesting C features:** direct terminal control (termios), VT100 escape handling, manual buffer/state management, compact monolithic implementation.

- **Size (cleaned set):** kilo.c ~1308 LOC.

I am planning to acquire more datasets so that i have at least 10, i am just giving them here to validate if i am headed in the correct direction with the datasets. **NOTE:** i have a dataset cleaning step where i remove a lot of the redundant files in the repos just to make it easier for the translation agent to translate it without getting misled by the amount of input.

Potential strategies for APR

This part will outline some of the APR strategies that I can compare for my thesis. This is just a collection of different strategies. Which ones I chose to pick to compare in my experiment is something that I will have to discuss and nail down with my supervisors (Tibo and Welf). The strategies shown are mostly from literature that i have found and mixed together with my imagination of what i think would be fitting or interesting to compare. I have also roughly categorized the different strategies as best as i can. When referring to the source and target language respectively, i will only say *C* and *Rust*.

1. Primitive APR strategies (can be used for baseline)

These are the strategies that are meant to emulate how an average engineer of my level and expertise would debug using LLMs. These are not really scientifically interesting or cutting edge in any way but could serve as a good baseline. I will not include very detailed descriptions because one could easily infer the details of each strategy from the name of it.

1. Just giving raw Rust terminal errors output from the compiler
2. Raw rust errors + target code
3. Raw rust errors + target code + source code for context

2. More interesting APR strategies

These strategies are strategies that would require some kind of engineering beyond just feeding an LLM some compiler outputs which would be interesting to compare from a research perspective.

1. Error-family guided context

- *Input*: compiler output + detected error family tag (macro, unresolved_symbol, type, borrow, syntax)
- *How*: add a lightweight classifier before prompting
- *Why*: reduces noise and makes the model focus on the real root cause first

2. Compiler-structured prompting

- Use **rustc --error-format=json** instead of raw text.
- Feed the LLM structured fields: code, span, suggested_replacement, label.
- This is very state-of-the-art-ish and often beats plain terminal logs.

3. Diagnose-then-patch

- *First call*: produce structured diagnosis (root cause, target symbols, fix plan).
- *Second call*: patch only.
- Helps reduce chaotic edits and error cycling.

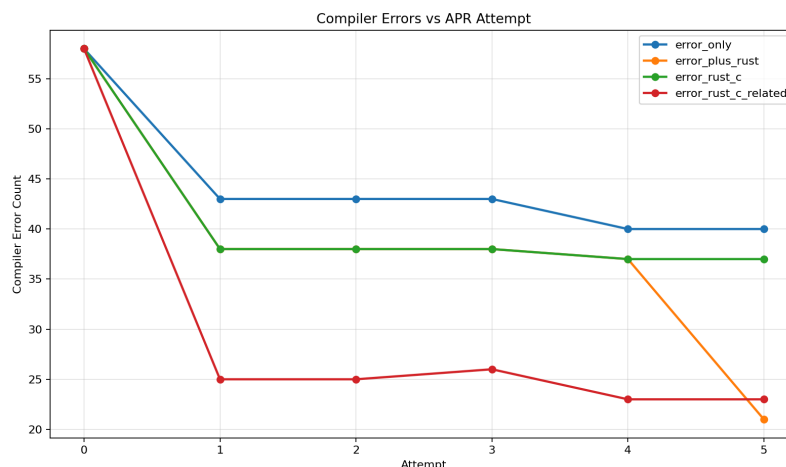
4. Memory

- *Input*: S3 + summary of previous attempts (what changed, error delta, what failed).
- *How*: append short structured memory block each iteration.
- *Why*: prevents looping on same failed fix pattern; good for plateau cases like SDS macro errors.

Potential metrics to evaluate/calculate

The metrics that i think are most relevant and interesting to evaluate for the study is the following:

- **Compile success rate**: the ratio of runs where ‘cargo check’ passes within max attempts
- **Error reduction per attempt**: Track compiler error over number of attempts. I was already tracking this for my pilot experiment and it allows me to create intuitive plots like this:



- **“Normalized” error reduction:** maybe i can compare the error reduction across several datasets, not just do it for each one.
- **Area Under Error Curve**
- **Token cost and time:** would be interesting to calculate some efficiency and cost metrics to allow for readers to evaluate if error reduction is worth it if its way more expensive. This is more of a secondary metric in my opinion though and not necessary.
- **Patch size and edit volume:** see how many LOC's are changed to see if one strategy is more aggressive in its repairs then others.